

Tarea: Algoritmos de Exclusión Mutua con Monte Carlo (Test-and-Set)

Autor: Cesar Raul Ramirez Cab 69967

Fecha: 28/09/2026

Explicación del Algoritmo Implementado

Para proteger la variable compartida puntosDentro ante accesos concurrentes, implementé el algoritmo de exclusión mutua **Test-and-Set** utilizando la función atómica nativa del compilador `__atomic_test_and_set`.

El funcionamiento del programa se divide en los siguientes pasos:

1. **Evaluación Local:** Cada hilo genera coordenadas aleatorias (x, y) dentro del rango $[-1.0, 1.0]$ y verifica la condición del círculo unitario:

$$2. \quad x^2 + y^2 \leq 1.0$$

3. **Entrada a la Sección Crítica (lock_test_and_set):** Si el punto cae dentro del círculo, el hilo intenta adquirir el cerrojo mediante una instrucción atómica e indivisible a nivel de hardware:
 - Si `lock_flag` se encuentra en 0 (desbloqueado), la función lo cambia atómicamente a 1 y el hilo entra inmediatamente a la sección crítica.
 - Si `lock_flag` ya vale 1 (bloqueado por otro hilo), el hilo entra en un bucle de **espera activa (spin lock)**, evaluando continuamente la variable hasta que quede libre.
4. **Sección Crítica:** El hilo incrementa de forma segura el contador global:
5. C

None

```
puntosDentro++;
```

- 6.
7. **Salida de la Sección Crítica (unlock_test_and_set):** El hilo ejecuta `__atomic_clear(&lock_flag, __ATOMIC_RELEASE)`, reajustando la variable a 0 para habilitar el acceso al siguiente hilo en espera.

Conclusión y Análisis de Rendimiento

A partir de las pruebas experimentales ejecutadas con **100,000,000 de puntos**, obtuve los siguientes resultados:

- **2 hilos:** 7.210 s ($\pi = 3.141659$)
- **4 hilos:** 8.414 s ($\pi = 3.141503$)
- **8 hilos:** 22.493 s ($\pi = 3.141743$)
- **16 hilos:** 44.498 s ($\pi = 3.141892$)

Observaciones Principales:

- **Degradación del rendimiento por contención extrema:** Aumentar la cantidad de hilos no mejoró el tiempo de ejecución; por el contrario, lo empeoró notablemente (la prueba de 16 hilos fue **6.1 veces más lenta** que la de 2 hilos).
- **Frecuencia de sincronización de grano fino:** Dado que el cerrojo se solicita e incrementa directamente dentro del bucle de cada punto dentro del círculo, el sistema debe gestionar la entrada y salida de la sección crítica aproximadamente **78.5 millones de veces**.
- **Sobrecosto de la espera activa (*Spinning*):** Al haber múltiples hilos compitiendo al mismo tiempo por la misma posición de memoria, la mayor parte del tiempo de procesador se malgasta ejecutando el bucle while de espera activa, saturando los núcleos de la CPU en lugar de realizar trabajo útil.
- **Corrección matemática:** A pesar de la pérdida de eficiencia por la sincronización frecuente, la estimación de π se mantuvo precisa en todas las corridas (≈ 3.1415), lo que confirma que el mecanismo de Test-and-Set evita con éxito las condiciones de carrera (*race conditions*).